Caddie

Testing Report

Tadhg Donnelly | 22480084 | tadhg.donnelly6@mail.dcu.ie

Gavin Szumlicki | 22511509 | gavin.szumlicki2@mail.dcu.ie

Boualem Benatallah | boualem.benatallah@dcu.ie

30/04/2026

1. Introduction

This document outlines the testing carried out for Caddie and the results obtained. Its purpose is to show that the system was tested appropriately and that the software is of sufficient quality for use. The report is divided into two main sections: traditional testing, which covers code-focused testing carried out during development, and user testing, which covers the feedback gathered from users, the findings identified, and the changes made in response.

2. Testing Strategy

The testing strategy for Caddie was designed to reflect the structure of the system and the different types of risk across the project. Because Caddie includes a web frontend, backend API, background worker processes, and AI-supported features, no single testing method was sufficient on its own. For that reason, multiple testing layers were used, including ad hoc testing during development, formal unit and integration testing, end-to-end testing, and user testing. Manual ad hoc testing was useful for quickly checking features as they were built, while formal testing provided defined test cases with expected outcomes that could be compared against actual results. Together, these approaches helped establish that the system behaved correctly, supported key user workflows, and met an appropriate standard of software quality. In addition, automated checks in the CI pipeline provided further confidence that changes did not introduce regressions by running core validation steps consistently as the project evolved.

3. Test Environment and Tools

The test environment for Caddie reflected the project's full architecture, including the Next.js web application, FastAPI backend, Celery worker, Redis queue, and external services such as Supabase and OpenAI. Different tools were used across the stack, including Pytest for backend and worker testing, Vitest for frontend component testing, and Playwright for end-to-end browser testing. Local testing was carried out using the development stack with the web app, API, worker, and supporting services running together where required, while lighter tests were run in isolation where possible. Automated checks were also integrated into the CI pipeline to run linting, type-checking, unit and integration tests, and browser-based test flows, helping to catch regressions before deployment. This was supported by a deployment process that also included health verification, making testing part of both development and deployment confidence.

4. Traditional Testing

Traditional testing in Caddie focused on verifying the technical correctness of the system across the main layers of the application. A combination of ad hoc testing, formal automated tests, and browser-level testing was used to confirm that core features behaved as intended and that regressions were identified as development progressed. Formal testing was used alongside ad hoc testing to provide more structured and repeatable evidence of software quality. These tests were designed with defined scenarios, expected results, and actual observed outcomes, allowing behaviour to be assessed more objectively. In Caddie, formal testing included unit tests, integration tests, component tests, and end-to-end tests, each targeting a different level of the system.

4.1 Ad Hoc Testing During Development

Ad hoc testing was carried out continuously during development as new features were implemented and existing features were refined. This included manually checking interface behaviour, verifying API responses during development, testing upload and ingestion flows, and confirming that fixes resolved the intended issue without introducing obvious regressions. While this was useful for rapid feedback and early bug detection, ad hoc testing alone was not sufficient, as it did not always provide repeatable evidence or clearly defined expected outcomes.

4.2 API Unit and Route Testing

API unit and route testing was used to verify backend behaviour in isolation, particularly around request handling, validation, permissions, status codes, and response structures. These tests were appropriate for the FastAPI layer because much of Caddie's core logic depends on reliable backend behaviour, including chat, hubs, memberships, FAQs, and source management. By testing routes and backend helpers with mocked external dependencies, it was possible to confirm that the API behaved correctly without requiring the full deployed stack to be running.Many of the tests were designed before implementation, and the majority were purposefully kept short and simple - each test should test 1 specific scenario. In total, the API test suite contains 243 tests, of which 238 are unit and route tests and 5 are integration tests. The output shown below presents the results of running these tests and provides evidence that the backend behaved as expected across the main API features.

```
243 passed, 17 warnings in 3.55s
```

4.3 API Integration Testing

API integration testing was used to verify that multiple backend modules worked together correctly through the real FastAPI application. This was especially useful for request flows that crossed route handlers, dependency wiring, and store or queue boundaries. Compared with isolated unit tests, integration tests provided greater confidence that complete backend request handling still behaved correctly when the application was exercised more realistically. This helped validate that important server-side flows continued to function as expected beyond the level of individual units.

4.4 Worker Module Testing

Worker module testing focused on the background-processing logic used for ingestion, extraction, chunking, transcription, and related helper behaviour. This was important because Caddie relies on asynchronous processing for uploaded documents, web sources, YouTube sources, and reminder-related features. Testing at this level allowed the worker's logic to be validated without depending on live network calls or full external services, making it possible to check complex helper behaviour in a controlled and repeatable way. At the time of writing, the worker test suite contains 39 tests. The output shown below provides evidence of these tests being executed and demonstrates the pass/fail results for the worker module.

```
39 passed, 12 warnings in 8.83s
```

4.5 Web Component Testing

Web component testing was used to verify isolated frontend behaviour, including rendering, state changes, conditional UI, and user interaction within individual components. This was appropriate for the Next.js frontend because many usability issues and regressions can occur at component level before they become full system-level failures. Component tests made it possible to check interface logic in a lightweight environment, while still providing stronger evidence than manual checking alone. At the time of writing, the web component test suite contains 150 tests. The output shown below provides evidence of these tests being executed and demonstrates the pass/fail results for the frontend component layer.

```
Test Files  29 passed (29)
     Tests  150 passed (150)
  Start at  13:53:38
  Duration  49.42s
```

4.6 End-to-End Testing

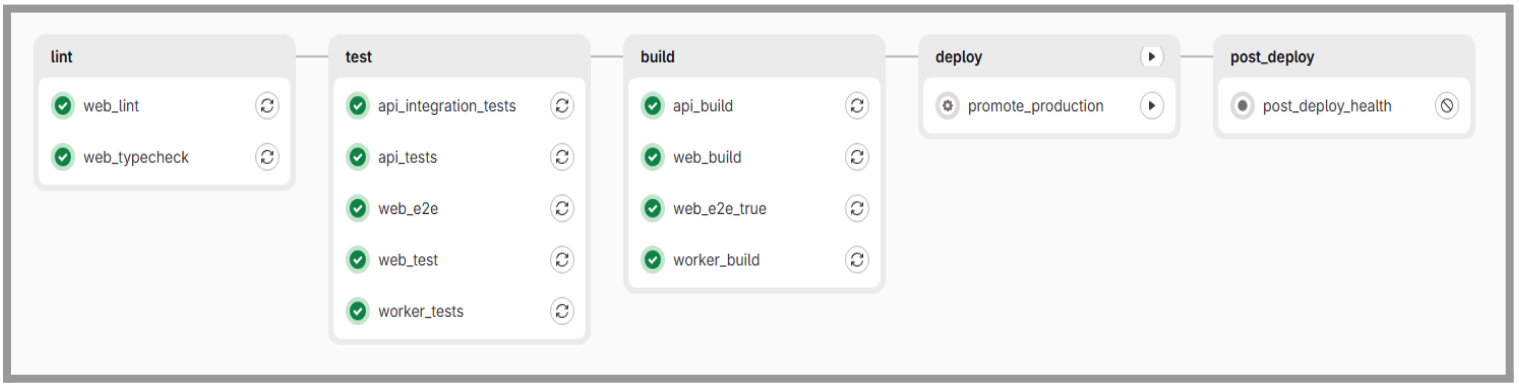
Mocked end-to-end tests were used to exercise real browser journeys through the web application while replacing external dependencies such as authentication and API traffic with controlled test doubles. This made the tests more stable and suitable for continuous integration, as they could validate route-level and UI-level behaviour without requiring the entire backend stack or third-party services to be live. These tests were useful in earlier stages for checking important user flows in a repeatable and CI-friendly way.

True end-to-end tests were used for the highest-confidence checks, running the real web application against the real local stack, including the API, worker, Redis, and external integrations where required. These tests were more expensive to run, but they were valuable for validating full-stack workflows such as ingestion, retrieval, and chat behaviour under realistic conditions. This layer was especially useful for acceptance-style verification and release confidence, as it tested the system much closer to how it would behave in actual use. Both of these layers are added to the GitLab CI Pipeline

5. CI/CD

Caddie's CI/CD setup was implemented through GitLab CI, with separate stages covering linting, testing, and build validation across the different applications. The pipeline includes frontend linting and type checking, web tests, backend and worker tests, build verification, and higher-level end-to-end validation. This provided an automated way to catch regressions before changes reached the main branch and helped maintain consistency across a codebase that spans several runtimes and services.

Deployment was implemented as a controlled promotion flow, with promote_production acting as a manual gate for deployment. GitLab acts as the primary source repository and CI control point, while GitHub is used as a deployment mirror for the hosted services. The frontend is deployed through Netlify, while the API, worker, and scheduler processes are deployed through Railway. Once successfully deployed via the promote_production job, a post_deploy_health job runs to verify the live services.



6. User Testing

6.1 Objectives

The objective of user testing was to evaluate Caddie from the perspective of real users carrying out realistic onboarding and knowledge-retrieval tasks. While traditional testing was used to verify technical correctness, user testing focused on ease of use, clarity of workflows, and the usefulness of the system's core features in practice. In particular, the sessions aimed to assess whether users could successfully navigate hubs, upload and manage sources, retrieve grounded answers through chat, inspect citations, use generated FAQs and guides, interact with reminders, and understand moderation-related functionality. A further objective was to identify usability issues, points of confusion, and any bugs that had not been identified during development.

6.2 Methodology

User testing was conducted as a task-based evaluation using a structured test script. Participants were asked to complete three scenarios. Scenario A focused on using a pre-built hub to retrieve information and interact with existing content. Scenario B required participants to create and configure a new hub from scratch, upload the provided sources, monitor ingestion, and then use the hub for retrieval tasks. Scenario C allowed participants to explore the application more freely and provide broader feedback on usefulness, limitations, and overall experience.

Participants were encouraged to complete tasks as naturally as possible and to note anything confusing, frustrating, or unclear. They were also told that there were no right or wrong actions, as the purpose of the sessions was to evaluate the system rather than the user. If participants became stuck, they were allowed to consult the user manual and continue, while still reporting any difficulties experienced. After completing the scenarios, participants were asked to complete a questionnaire and provide additional feedback.

6.3 Participant Overview

The participants in the user-testing sessions were selected to provide feedback relevant to Caddie's intended use as an onboarding and knowledge-support tool. Since the system is designed for users working with structured documentation, onboarding materials, and information retrieval tasks, participants were chosen from backgrounds where these kinds of workflows would be familiar and meaningful. A total of 8

participants took part in the user-testing sessions, with varying levels of technical proficiency.

6.4 Session Tasks

The session tasks were divided into three scenarios. In Scenario A, participants were given access to a hub that had already been created and populated with sources. They were asked to accept the hub invite, inspect the hub contents, identify the number and types of sources available, and locate the source most relevant to workstation setup. They then used chat to answer specific onboarding questions from the sources already present in the hub. Participants were also asked to flag a chat response, inspect citations, manually verify one answer against the original source, generate an FAQ and a guide, create a manual reminder, and review flagged chat items in the admin panel.

In Scenario B, participants created a new hub from scratch for engineering onboarding resources. They provided a name and description, uploaded the six local documents, added the provided web source and YouTube source, and monitored ingestion until sources became ready for use. They then reviewed the source list, identified different source types, and used chat to answer a second set of onboarding questions. Participants also reviewed suggested reminders, accepted or denied them based on relevance, created a manual reminder, and reflected on which feature was most helpful and which part of the setup or usage flow felt confusing or slow. In Scenario C, participants were invited to explore the application more freely. This gave them the opportunity to use whichever features they found most relevant and to provide broader comments on what felt useful, what was frustrating, what seemed to be missing, and their overall impression of the system.

6.5 Feedback and Findings

The structure of the user-testing sessions made it possible to gather feedback across both guided and open-ended usage. Scenario A primarily evaluated retrieval quality, citation trustworthiness, reminders, FAQs, guides, and moderation workflows in a prepared environment. Scenario B focused more heavily on first-time setup, source upload, ingestion visibility, and end-to-end usability from hub creation through to answering questions. Scenario C complemented these structured tasks by allowing issues or preferences to emerge more naturally during free exploration. The feedback gathered from these sessions was analysed qualitatively and grouped into recurring

themes. The feedback was evaluated, and acted on where appropriate. Alongside this, User Testing surfaced any potential errors associated with the hosted project.

7. Results

7.1 Traditional Testing Results

The traditional testing carried out for Caddie produced strong evidence that the main technical layers of the system were functioning correctly. At the time of writing, the backend API suite contained 243 tests in total, comprising 238 unit and route tests and 5 integration tests. These tests covered core backend areas such as hubs, chat, memberships, FAQs, reminders, moderation, and source-management behaviour. The worker module test suite contained 39 tests, focused on ingestion, extraction, chunking, transcription, and related helper logic. The web component test suite contained 151 tests, providing coverage for important frontend behaviour and user interaction at component level.

In addition to these lower-level tests, browser-based end-to-end testing was also used. Mocked Playwright tests provided stable validation of key web journeys in CI, while true end-to-end tests were used for higher-confidence checks against the real local stack. Together, these results indicate that Caddie was tested across backend, worker, frontend, and full user-flow levels rather than relying on only one form of testing. The CI pipeline further strengthened this by ensuring that linting, type-checking, testing, build validation, and deployment health checks were run consistently before promotion.

7.2 User Testing Results

User testing was carried out across 8 sessions using the structured three-scenario format described in the previous section. These sessions produced useful results in two main areas: first, they showed that users were generally able to complete the intended core workflows of the system, including joining hubs, retrieving information through chat, checking citations, generating FAQs and guides, and using reminders; second, they exposed several usability issues and real product problems that required changes.

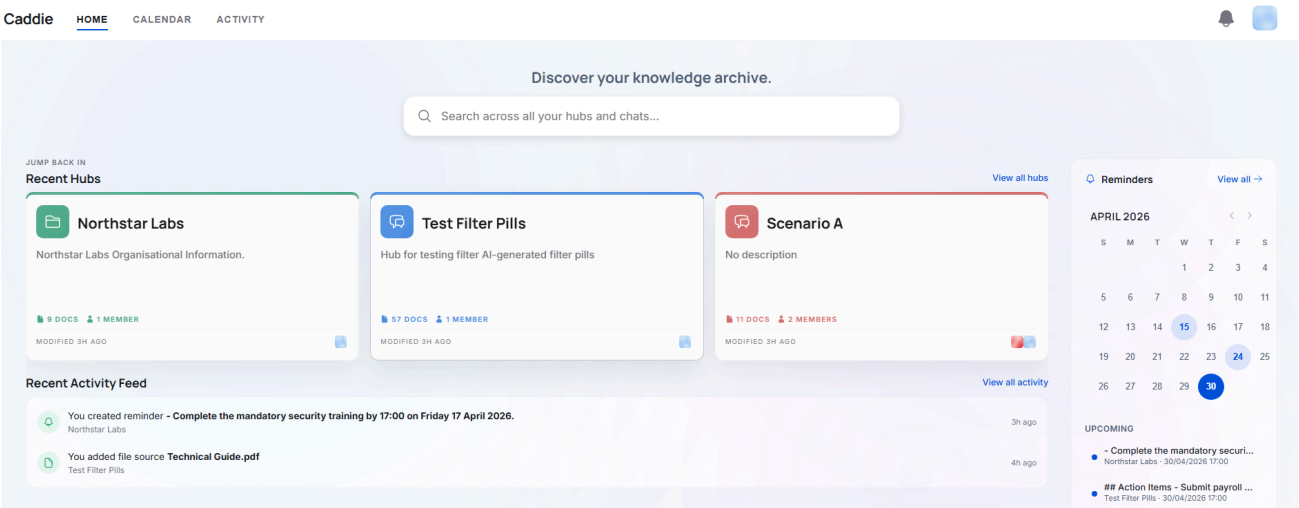
A clear result from the sessions was that users found the overall concept of hub-based onboarding support useful, particularly the ability to ask direct questions over uploaded material and trace answers back to citations. However, the sessions also showed that some areas of the interface needed to be clearer and more visually readable. Feedback

repeatedly pointed toward adding some more colour to the UI, making the system feel easier to scan, and exposing more useful content earlier in the user journey.

The most important technical problem identified during user testing concerned YouTube ingestion. In practice, some test sessions encountered failures caused by YouTube rate-limiting and anti-bot restrictions. This was significant because YouTube support formed part of the intended ingestion workflow in Scenario B, and failures here directly affected the user testing experience. Since this issue arose in realistic usage rather than only in isolated development conditions, it became an important result of the user-testing process.

7.3 Changes Made Following Feedback

The user-testing sessions led to both UI improvements and deeper functional changes. One set of changes focused on the interface itself. Based on participant feedback, the visual design was improved through the introduction of stronger and more deliberate colour usage, helping the application feel clearer and less flat to navigate. Another notable change was made to the homepage hub display, where the number of hubs shown was increased from 2 to 3. This responded to feedback that users wanted more immediate visibility of their available hubs without needing to navigate further into the interface. These were relatively small changes from a technical perspective, but they were important because they directly addressed how users experienced the system.



Another set of changes addressed bugs identified during testing. One important issue involved citation behaviour. During user testing, some responses were returned without citations when users expected supporting references to be present.. Feedback made it clear that citation visibility and source inspection were central to user trust, particularly because Caddie is intended to support grounded answers over uploaded material. In response, the citation flow was hardened so that grounded answers and their supporting references were handled more reliably, and additional attention was given to making source verification more consistent during normal use.

The most substantial change was the response to YouTube ingestion failures. During testing, the original approach depended on obtaining transcripts directly from YouTube videos. When failures began appearing due to rate-limiting and bot-detection behaviour, an attempt was first made to improve reliability through configuration-based fixes such as cookie support and related hosted-worker adjustments. Although this improved some cases, it did not fully resolve the issue, and it was not reliable enough to support a smooth user-facing workflow.

At that point, we had to assess alternative solutions. One possibility was to continue pursuing direct transcript extraction through other libraries or proxy-based approaches, such as combinations involving yt-transcript-api. However, those options introduced further uncertainty, operational complexity, and dependence on brittle third-party behaviour outside our control - we may spend time implementing these new apis just for them to break.

The alternative was to provide a fallback where users could supply media (MP3s, MP4s) themselves and Caddie would handle the transcription pipeline internally. This second option was chosen because it gave the project a more dependable and controllable recovery path. As a result, manual media upload was introduced as a fallback for recoverable YouTube failures, allowing users to upload downloaded audio or video and have Caddie transcribe it directly. This was a pragmatic decision informed by user testing: instead of preserving a fragile feature at all costs, the system was adapted so that the user could still complete the task reliably.

7.4 Evaluation of Overall Software Quality

Taken together, the testing suggests that Caddie reached a good overall standard of software quality for the project's intended scope. The automated test suites provide strong evidence that important technical behaviour was checked systematically across multiple layers, while the user-testing sessions showed that the system could support realistic onboarding and knowledge-retrieval tasks in practice. The user-testing results were especially important because they demonstrated that quality was not judged only in terms of code correctness. We used these sessions to identify real issues, evaluate what users found confusing or useful, and make changes where appropriate. This strengthens the quality of the final system, as it shows an iterative development process in which user feedback materially influenced the product.

The response to the YouTube ingestion issue is a particularly strong example of this. Rather than ignoring the failure or treating it as an edge case, the issue was investigated, alternative approaches were assessed, and a more reliable fallback mechanism was introduced. This reflects a realistic and responsible approach to software quality: not all problems can be eliminated at source, but important user-facing failures should be handled in a way that preserves the usefulness of the system.

7.5 Limitations and Remaining Issues

Despite the positive results, there are still some limitations. First, user testing was conducted on a limited scale, with 8 sessions rather than a large or statistically broad participant base. This means the findings are valuable qualitatively, but they should not be treated as exhaustive evidence of how all users would behave. Second, some features, especially those involving third-party services such as YouTube, remain partly constrained by the behaviour of external platforms. Although the manual media fallback substantially improved resilience, it does not completely remove the limitations of relying on external content platforms. YouTube link ingestion is still a functioning feature, but is subject to some unavoidable intermittent failures.

There are also broader limits to what automated and scenario-based testing can prove for an AI-assisted system. Caddie's chat, generated FAQs, guides, and reminder suggestions can be tested for grounding, structure, and workflow correctness, but user perception of answer quality and usefulness still requires human evaluation. For that reason, the combination of traditional testing and user testing was necessary. Even so, continued refinement would still be beneficial, particularly around polishing real uise

cases, improving UI clarity further, and continuing to harden any functionality that depends on third-party integrations.